

Module 1 — Core Mastery

Lesson 1.2

Files, search, and the right tool for the job

Claude has dedicated tools for reading, editing, and searching. Knowing which it picks (and nudging it when wrong) is half of fast iteration.

Why it matters

When Claude reaches for `cat`, `find`, `grep`, `sed`, or `awk` through Bash, three things go wrong: it costs more tokens, it's slower, and you lose structured information Claude could have gotten from a typed tool. The dedicated tools are not just nicer — they're how Claude is meant to work.

The tool taxonomy

Goal	Right tool	Wrong (but common)
Read a file you know the path of	Read	<code>cat</code> , <code>head</code> , <code>tail</code>
Find files by name pattern	Glob (e.g. <code>src/**/*.tsx</code>)	<code>find</code> or <code>ls -R</code>
Search file contents	Grep (ripgrep under the hood)	<code>grep -r</code> via Bash
Make a small surgical change	Edit	<code>sed -i</code>
Create or fully rewrite a file	Write	<code>echo > file</code>
Open-ended multi-round search	Agent with Explore subagent	Many Grep calls

You don't usually pick the tool directly — Claude does. But you can steer it:

"Use Grep to find every call site of `parseConfig`, don't shell out."

"Use Glob to list the migration files instead of `ls`."

If you find yourself watching Claude `cat` a file, interrupt and remind it.

Glob patterns that pay off

- `**/*.ts` — all TypeScript files at any depth
- `src/**/*.ts,tsx` — limit to a subtree with multiple extensions
- `**/*.test.*` — every test file regardless of language
- `migrations/2026*.sql` — date-prefixed slices

Grep idioms

```
# Find a function and show line numbers
"Grep for `function createUser` with -n"
```

```
# Get just file paths
"Grep for TODO across the repo, files-with-matches mode"
```

```
# Multi-line patterns
"Grep with multiline:true for `interface Foo \\\\[\\s\\S]*?bar`"
```

Grep is the right tool for symbol lookup. Reading the whole file to find a function is a context tax — Grep narrows it first.

Reading large files

By default `Read` returns the full file. For very large files (>2000 lines), pass `offset` and `limit`:

```
"Read dist/bundle.js lines 5000–5200."
```

Better: ask Grep for the lines of interest, then `Read` a tight window around them.

Editing rules of thumb

- **Read before you Edit.** The Edit tool requires that the file has been read in the session.
- **`old\_string` must be unique** in the file. If it isn't, expand the surrounding context until it is, or use `replace_all`.
- **Never use Edit to delete a file.** Use a shell command (`rm`) — with confirmation.
- **For full rewrites, use Write.** Don't try to Edit your way through more than ~5 changes — start over.

Search depth: when to delegate to an Explore agent

If the search will take more than ~3 query rounds, spawn the `Explore` subagent:

```
"Use the Explore agent to find every place that touches user permissions across the backend and frontend. Report the top 10 hot spots."
```

You pay one agent invocation; you get a synthesized report without those rounds polluting your main context window. We'll cover this thoroughly in Module 4.

Try it

In any nontrivial repo:

- 1 Ask Claude to find a function definition without specifying how. Watch what tool it uses.
- 2 Ask it to find every call site of that function "using Grep, files-with-matches mode."
- 3 Compare the speed and clarity of the two attempts.
- 4 Ask it to rename a constant across the codebase. Note whether it uses Grep first, then Edit per-file, vs. trying to do it in one shot.

Gotchas

- **`Grep` is ripgrep, not POSIX grep.** Literal braces and parens need escaping (`interface\{\}` to match Go's `interface{}`).
- **Glob is sorted by mtime, not alphabetically.** If you need alphabetical, sort it yourself.
- **Edit can fail confusingly** if invisible characters differ (tabs vs spaces, CRLF vs LF). Use `Read` first to see exactly what's there.
- **`Write` overwrites without warning.** Make sure you actually want to replace, not edit.

Further reading

- 1.5 Context management — why search choices affect context
- 4.1 Subagents overview — when to delegate search

Next

→ 1.3 Slash commands you'll actually use